

NAME : SAMIKSHA SHIVAJI BHOSALE

PRN : 202501050011

BATCH : CH1

PRACTICAL 1

PRACTICAL 1.1.1 CALCULATE MOMENTUM

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

CODE :

```
mass=float(input())
velocity=float(input())
momentum=mass*velocity
print(f"{momentum:.2f}kgm/s")
```

CODETANTRA

Home Learn Anywhere

202501050011@mitaoe.ac.in Support Logout

1.1.1. Calculate Momentum

03:18

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

- m is the mass of the object (in kilograms).
- v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

Test case 1

5.0

10.0

50.00kgm/s

calculate...

Submit

```

1 mass=float(input())
2 velocity=float(input())
3 momentum=mass*velocity
4 print(f"momentum: .2f)kgm/s")
5
6

```

Average time

0.011 s

10.50 ms

Maximum time

0.016 s

16.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

10 ms

Debug

Expected output

5.0

10.0

50.00kgm/s

Actual output

5.0

10.0

50.00kgm/s

Test case 2

9 ms

Terminal Test cases

< Prev

Reset

Submit

Next >

PRACTICAL 1.1.2 CONDITIONAL CALCULATION BASED ON THE NUMBER OF DIGITS

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

$$1 \leq \leq 999$$

Input Format:

- The input consists of a single integer .

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

CODE :

```

import math

number=int(input())

if 1<=number<=9:

    print(number*number)

elif 10<=number<=99:

    print(f"{math.sqrt(number):.2f}")

elif 100<=number<=999:

    print(f"{math.cbrt(number):.2f}")

else:

    print("Invalid")

```

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:
 • The input consists of a single integer n .

Output Format:
 • If n is a single-digit number, print its square.
 • If n is a two-digit number, print its square root (rounded to two decimal places).
 • If n is a three-digit number, print its cube root (rounded to two decimal places).
 • Else print "Invalid".

Sample Test Cases

Test case	Input	Expected output	Actual output
Test case 1	9	81	81
Test case 2	166		

Code Editor:

```

1 #write your code here...
2 import math
3 number=int(input())
4 if 1<=number<=9:
5     print(number*number)
6 elif 10<=number<=99:
7     print(f"{math.sqrt(number):.2f}")
8 elif 100<=number<=999:
9     print(f"{math.cbrt(number):.2f}")
10 else:
11     print("Invalid")

```

Test Results:
 Average time: 0.005 s (4.57 ms) | Maximum time: 0.006 s (6.00 ms)
 4 out of 4 shown test case(s) passed
 3 out of 3 hidden test case(s) passed

PRACTICAL 1.1.3 DAYS BETWEEN TOW DATES

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format .

- The second line contains the second date in the format .

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

CODE :

```
from datetime import date
```

```
from datetime import datetime
```

```
date1 = input().strip()
```

```
date2 = input().strip()
```

```
d1 = datetime.strptime(date1,"%Y-%m-%d")
```

```
d2 = datetime.strptime(date2,"%Y-%m-%d")
```

```
difference = d2 - d1
```

```
print(difference.days)
```

The screenshot shows the CodeTANTRA online IDE interface. On the left, the problem description for "1.1.3. Days between Two Dates" is displayed, including input/output formats and sample test cases. The main editor shows the Python code for calculating the difference in days between two dates using the `datetime` module. The code is as follows:

```
1 from datetime import date
2
3 #write your code here...
4 from datetime import datetime
5 date1 = input().strip()
6 date2 = input().strip()
7 d1 = datetime.strptime(date1,"%Y-%m-%d")
8 d2 = datetime.strptime(date2,"%Y-%m-%d")
9 difference = d2 - d1
10 print(difference.days)
11
```

On the right, the test results are shown. The average time is 0.034 s and the maximum time is 0.101 s. Two out of two shown test cases passed, and two out of two hidden test cases passed. The test cases are:

Test Case	Expected Output	Actual Output
Test case 1	2014-07-02 2014-11-02 123	2014-07-02 2014-11-02 123
Test case 2	2014-07-02 2014-07-11	2014-07-02 2014-07-11

PRACTICAL 1.1.4 REVERSE A NUMBER

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

CODE :

```
num=int(input())
```

```
reversed_num=int(str(num)[::-1])
```

```
print(reversed_num)
```

The screenshot displays the CODETANTRA online IDE interface for the '1.1.4. Reverse a Number' problem. The left sidebar contains the problem description, input/output formats, and sample test cases. The main editor shows the Python code for reversing a number. The right sidebar displays the test results, indicating that 2 out of 2 shown test cases passed and 3 out of 3 hidden test cases passed.

Problem Description: Write a Python program to reverse the digits of the number and print the reversed number.

Input Format: The input is a single integer.

Output Format: The output prints a single integer, which is the reversed number.

Sample Test Cases:

Test case	Input	Expected output	Actual output
Test case 1	5367	7635	7635
Test case 2	0132	231	231

Code Editor:

```
1 #write your code here...
2 num=int(input())
3 reversed_num=int(str(num)[::-1])
4 print(reversed_num)
5
```

Test Results:

- Average time: 0.005 s, Maximum time: 0.006 s
- 4.60 ms, 6.00 ms
- 2 out of 2 shown test case(s) passed
- 3 out of 3 hidden test case(s) passed
- Test case 1 (5 ms) - Debug
- Test case 2 (4 ms)

PRACTICAL 1.1.5 MULTIPLICATION TABLE

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

CODE :

```
num=int(input())
```

```
for i in range(1,11):
```

```
    print(f"{num} x {i} = {num * i}")
```

The screenshot displays a coding platform interface for a problem titled "1.1.5. Multiplication Table". The problem description asks for a Python program that takes an integer as input and prints its multiplication table from 1 to 10. The input format specifies that the first line contains the integer. The output format shows the expected output for the input 8: "8 x 1 = 8", "8 x 2 = 16", "8 x 3 = 24", "8 x 4 = 32", and "8 x 5 = 40". The code editor shows the following Python code:

```
1 #write your code here...
2 num=int(input())
3 for i in range(1,11):
4     print(f"{num} x {i} = {num * i}")
5
6
```

The test results panel indicates that the code passed all test cases. It shows "2 out of 2 shown test case(s) passed" and "2 out of 2 hidden test case(s) passed". The average time taken is 0.010 s, and the maximum time is 0.017 s. The test case details show the expected output matching the actual output for the input 8.

PRACTICAL 1.2.1 PASS OR FAIL

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer , the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

CODE :

```
def cal(marks, total_courses):  
    if any(mark < 40 for mark in marks):  
        return "Fail"  
  
    total_marks = sum(marks)  
  
    aggregate_percentage = (total_marks / (total_courses * 100)) * 100  
  
    if aggregate_percentage > 75:  
        grade = "Distinction"  
  
    elif aggregate_percentage >= 60:  
        grade = "First Division"  
  
    elif aggregate_percentage >= 50:  
        grade = "Second Division"  
  
    elif aggregate_percentage >= 40:  
        grade = "Third Division"  
  
    return (aggregate_percentage, grade)  
  
num_courses = int(input())  
marks = list(map(int, input().split()))  
  
result = cal(marks, num_courses)  
  
if result == "Fail":  
    print("Fail")  
else:  
    aggregate_percentage, grade = result  
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
```



```
print(f"Grade: {grade}")
```

1.2.1. Pass or Fail

The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

Test case 1

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2

```
1 # write your code here
2 def cal(marks, total_courses):
3     if any(mark < 40 for mark in marks):
4         return "Fail"
5     total_marks = sum(marks)
6     aggregate_percentage = (total_marks / (total_courses * 100)) * 100
7     if aggregate_percentage > 75:
8         grade = "Distinction"
9     elif aggregate_percentage >= 60:
10        grade = "First Division"
11    elif aggregate_percentage >= 50:
```

Average time: 0.011 s, Maximum time: 0.017 s

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 (10 ms) Debug

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division

Test case 2 (10 ms)

Terminal Test cases

< Prev Reset Submit Next >

PRACTICAL 1.2.2 FIBONACCI SERIES

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

CODE :

```
def fibonacci(n,a=0,b=1):
```

```
    if n == 1:
```

```
        return a
```

```
    else:
```

```
        return fibonacci(n-1,b,a+b)
```

```
n = int(input())
```

```
for i in range(1, n + 1):
```

```
    print(fibonacci(i), end=" ")
```

1.2.2. Fibonacci Series 07:57

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

Test case 1

5

0 1 1 2 3

Test case 2

15

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Explorer fibonacci...

```
1 #def fibonacci(n):
2
3 # write the code..
4 def fibonacci(n,a=0,b=1):
5     if n == 1:
6         return a
7     else:
8         return fibonacci(n-1,b,a+b)
9
10
11 n = int(input())
```

Average time 0.007 s **Maximum time** 0.010 s

7.00 ms 10.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 7 ms **Debug**

Expected output 5

0 1 1 2 3

Actual output 5

0 1 1 2 3

Test case 2 6 ms

Terminal **Test cases**

< Prev Reset Submit Next >

PRACTICAL 1.2.3 PATTERN-1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

CODE :

```
n=int(input())
```

```
for i in range(1,n + 1):
```

```
    print("* " * i)
```

The screenshot shows the CODETANTRA online IDE interface. On the left, the problem description for '1.2.3. Pattern - 1' is visible, including input and output formats and sample test cases. The main editor shows a Python program that takes an integer input and prints a right-angled triangle pattern of asterisks. The program is submitted, and the results show that 2 out of 2 shown test cases and 4 out of 4 hidden test cases passed. The expected and actual outputs for Test case 1 are displayed, showing a right-angled triangle pattern of asterisks.

PRACTICAL 1.2.4 PATTERN-2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

CODE :

```
n=int(input())
```

```
for i in range(1,n+1):
```

```
    for j in range(1,i+1):
```

```
        print(j, end=" ")
```

```
    print()
```

The screenshot displays the CODETANTRA online coding environment. On the left, the problem statement for '1.2.4. Pattern - 2' is shown, requiring a Python program to print a right-angled triangle pattern of numbers. The input format specifies an integer for the number of rows, and the output format requires numbers separated by spaces. A sample test case is provided with input 5 and the expected output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

The main editor shows the following Python code:

```
1 # Type Content here...
2
3 n=int(input())
4 for i in range(1,n+1):
5     for j in range(1,i+1):
6         print(j, end=" ")
7     print()
8
```

Below the code editor, the execution results are displayed, showing that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The average time is 0.007 s and the maximum time is 0.008 s. A table compares the expected output with the actual output for Test case 1, showing a perfect match.

Expected output	Actual output
5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

At the bottom, there are buttons for 'Terminal', 'Test cases', 'Prev', 'Reset', 'Submit', and 'Next'.

